

Court of Claude — Proposal

Proposal: Warehouse (Expo + React Native) Petitioner

Mansour Ahmed The claim

Small auto-parts and small warehouses lose money in three boring ways: stock counts drift, items are hard to locate at the moment of sale, and pricing/quantity mistakes happen under time pressure. The proposal is a mobile-first warehouse workflow that makes those three failure modes harder to trigger by default. The product (what exists today)

A working Expo Router app with:

- Inventory tab for browsing and editing parts (including compatible car metadata)
- Barcode/part-number scanning flow to instantly find items
- Sale flow with cart count, checkout behavior, and receipt overlay
- Reports and settings screens as scaffolding for daily operations

Repository: <https://github.com/Mansur-ahmed7/React-native-warehouse> The wedge (why this wins)

The app is not trying to be an ERP. It is a "counter-speed" tool: it optimizes the single moment that causes the most damage-when a customer is waiting and the operator is guessing.

If scanning or searching fails, the operator makes up a number. If editing a part is slow, the operator avoids updating stock. If sales are not recorded at the moment they happen, reporting becomes fiction.

So the wedge is speed + minimal friction for:

- 1) locate the part
 - 2) confirm price/quantity
 - 3) record the sale
- The five-stage gauntlet (preemptive)

Because the court will execute weak ideas, here is the idea under five common failure tests. Stage 1 - Is the problem real?

Yes. Any shop that sells SKUs with similar names (filters, plugs, pads) routinely mis-picks and mis-prices. That loss is visible daily as:

- time wasted searching
 - stock mismatch
 - angry customers when the wrong part is sold
- Stage 2 - Is the solution meaningfully better than doing nothing?

Yes, if it reduces the time-to-correct-part at the counter and reduces "I'll update later" behavior.

The app targets these metrics:

- time from scan/search -> correct item selected
 - time from item selected -> sale recorded
 - percent of sales recorded same day
- Stage 3 - Can it be built and maintained?

Yes. The current implementation is already functional, built on a stable Expo stack. Data is stored locally (Zustand + AsyncStorage) to avoid infrastructure complexity during early validation. Stage 4 - Why won't it be copied instantly?

The defensibility is not the UI. It is the workflow density: the small decisions that remove friction (fast-add, recent scans, edit-in-place) and the local domain dataset (brands/models/parts) that reflects real shop language. Stage 5 - What kills it?

If the app fails at trust and speed, it dies.

- Trust: if the stock number is wrong, users stop using it.
- Speed: if scanning/searching is slower than memory, users stop using it.

Mitigation:

- Make edits fast and obvious.
- Keep the scanner flow resilient and predictable.
- Provide clear feedback (toasts) on actions. Near-term plan (2-4 weeks)
 - 1) Tighten the scanner loop (retry affordances, clearer not-found actions).
 - 2) Add simple CSV import/export for inventory backup.
 - 3) Add a minimal "daily close" report: sales total, items sold, low-stock list. What I'm asking the court to approve

Approval to submit this app as a credible, buildable product idea whose core value is counter-speed accuracy for small parts inventory and sales capture.

your honor.